

1. Dijkstra.c Dijkstra (다익스트라) 알고리즘

이 파일은 **다익스트라 알고리즘**을 구현합니다. 이 알고리즘은 가중 그래프에서 하나의 시작 정점으로부터 다른 모든 정점까지의 최단 경로를 찾는 데 사용됩니다. 간선 가중치는 음수가 아니어야 합니다.

주요 구성 요소:

- 자료구조:
 - **GraphType**: 인접 행렬을 사용하여 그래프를 표현하는 구조체입니다.
 - **n**: 그래프의 정점 수를 나타내는 정수입니다.
 - **weight[MAX_VERTICES][MAX_VERTICES]**: 정점 간의 간선 가중치를 저장하는 2차원 배열입니다. **INF** (예: 1000000)는 직접적인 경로가 없음을 나타냅니다.
 - **distance[MAX_VERTICES]**: 전역 정수 배열로, **distance[i]**는 시작 정점으로부터 정점 **i**까지 현재까지 알려진 최단 거리를 저장합니다. 시작 정점으로부터의 직접적인 간선 가중치로 초기화됩니다.
 - **found[MAX_VERTICES]**: 전역 불리언(정수) 배열로, **found[i]**가 **TRUE**(1)이면 정점 **i**에 대한 최단 경로가 확정되어 방문된 집합에 포함되었음을 의미하고, **FALSE**(0)이면 그렇지 않음을 의미합니다.
- 핵심 함수:
 - **int choose(int distance[], int n, int found[])**: 아직 **found** 집합에 포함되지 않았으면서 **distance** 값이 가장 작은 정점 **u**를 선택합니다. 이 정점의 인덱스를 반환합니다.
 - **void print_status(GraphType* g)**: 알고리즘의 각 단계에서 **distance**와 **found** 배열의 현재 상태를 출력하는 유틸리티 함수입니다. 알고리즘 진행 상황을 시각화하는 데 도움이 됩니다.
 - **void shortest_path(GraphType* g, int start)**: 다익스트라 알고리즘을 구현하는 주 함수입니다.
 1. **distance** 배열을 시작 정점에서 다른 모든 정점까지의 가중치로 초기화합니다.
 2. **found** 배열을 모든 정점에 대해 **FALSE**로 초기화합니다.
 3. **found[start]**를 **TRUE**로, **distance[start]**를 0으로 설정합니다.
 4. **g->n - 1**번 반복합니다 (또는 도달 가능한 모든 정점을 찾을 때까지):
 - **print_status()**를 호출하여 현재 단계를 표시합니다.
 - **choose()**를 호출하여 방문하지 않은 정점 중 가장 작은 거리를 가진 다음 정점 **u**를 선택합니다.
 - **u**를 **found**로 표시합니다 (**found[u] = TRUE**).
 - **u**의 각 이웃 **w**에 대해 아직 **found**가 아니라면:
 - 간선 (**u, w**)를 통해 **w**로 가는 경로(**distance[u] + g->weight[u][w]**)가 현재 **distance[w]**보다 짧으면 **distance[w]**를 업데이트합니다.
 - **int main()**:
 - 7개의 정점과 미리 정의된 간선 가중치를 가진 **GraphType** 구조체 **g**를 초기화합니다. **INF**는 직접적인 간선이 없음을 나타냅니다.
 - 정점 0에서 시작하는 최단 경로를 찾기 위해 **shortest_path(&g, 0)**를 호출합니다.

구현된 알고리즘:

다익스트라 알고리즘입니다. 이는 탐욕 알고리즘으로, 방문한 정점들의 집합(최단 경로가 알려진)을 유지하고 반복적으로 가장 가까운 미방문 정점을 추가합니다. 그런 다음 새로 추가된 정점의 이웃까지의 거리를 업데이트합니다.

코드 내 예시: `main` 함수에서 그래프를 정의하고 `shortest_path` 함수를 호출하는 부분을 통해 실제 사용 예시를 확인할 수 있습니다.

2. `Floyd.c` Floyd-Warshall (플로이드-워셜) 알고리즘

이 파일은 **플로이드-워셜 알고리즘**을 구현합니다. 이 알고리즘은 가중 그래프에서 모든 정점 쌍 간의 최단 경로를 찾는 데 사용됩니다. 양수 또는 음수 가중치를 가진 간선을 처리할 수 있지만, 음수 사이클은 처리할 수 없습니다.

주요 구성 요소:

- **자료구조:**
 - `GraphType`: `Dijkstra.c`와 유사하게 인접 행렬을 사용하여 그래프를 나타냅니다.
 - `n`: 정점의 수입니다.
 - `weight[MAX_VERTICES][MAX_VERTICES]`: 간선 가중치를 저장하는 인접 행렬입니다. `INF(1000000)`는 직접적인 경로가 없음을 나타냅니다.
 - `A[MAX_VERTICES][MAX_VERTICES]`: 전역 2차원 정수 배열입니다. 알고리즘 완료 후 `A[i][j]`는 정점 `i`에서 정점 `j`까지의 최단 거리를 저장합니다.
- **핵심 함수:**
 - `void printA(GraphType *g)`: 거리 행렬 `A`의 현재 상태를 출력하는 유틸리티 함수입니다. `INF` 값에 대해서는 `*`를 표시합니다.
 - `void floyd(GraphType* g)`: 플로이드-워셜 알고리즘을 구현하는 주 함수입니다.
 1. `g->weight` 행렬을 복사하여 `A` 행렬을 초기화합니다. 이는 초기 최단 경로를 직접 간선으로 설정합니다.
 2. `printA(g)`를 호출하여 초기 `A` 행렬을 표시합니다.
 3. `k`를 0부터 `g->n - 1`까지 반복합니다 (`k`는 중간 정점을 나타냄).
 - 각 정점 쌍 (`i, j`)에 대해:
 - `k`를 경유하는 `i`에서 `j`까지의 경로(`A[i][k] + A[k][j]`)가 현재 알려진 최단 경로 `A[i][j]`보다 짧은지 확인합니다.
 - 더 짧으면 `A[i][j]`를 업데이트합니다.
 - `k`의 각 반복 후 `printA(g)`를 호출하여 `A` 행렬이 어떻게 변화하는지 보여줍니다.
 - `int main()`:
 - 7개의 정점과 미리 정의된 간선 가중치를 가진 `GraphType` 구조체 `g`를 초기화합니다.
 - 모든 쌍의 최단 경로를 계산하기 위해 `floyd(&g)`를 호출합니다.

구현된 알고리즘:

플로이드-워셜 알고리즘입니다. 이는 동적 프로그래밍 알고리즘입니다. 각 정점 쌍 (`i, j`)에 대해 가능한 모든 중간 정점 `k`를 고려하고 `k`를 통해 더 짧은 경로가 발견되면 최단 경로를 업데이트합니다.

코드 내 예시: `main` 함수는 7개 정점을 가진 그래프를 정의하고 `floyd` 함수를 실행하여 모든 쌍의 최단 경로를 계산하는 예시를 보여줍니다.

3. `Kruskal.c` Kruskal (크루스칼) 알고리즘

이 파일은 **크루스칼 알고리즘**을 구현합니다. 이 알고리즘은 연결된, 무방향, 가중 그래프에 대한 최소 신장 트리 (MST)를 찾습니다. MST는 사이클을 형성하지 않으면서 최소 총 간선 가중치로 모든 정점을 연결하는 하위 그래프입니다.

주요 구성 요소:

- **자료구조:**

- `parent[MAX_VERTICES]`: 분리 집합 자료구조(Union-Find)를 구현하는 데 사용되는 전역 정수 배열입니다. `parent[i]`는 원소 `i`의 부모를 저장합니다. `parent[i] == -1`이면 `i`는 해당 집합의 루트입니다.
- `struct Edge`: 간선을 정의합니다.
 - `start`: 간선의 시작 정점입니다.
 - `end`: 간선의 끝 정점입니다.
 - `weight`: 간선의 가중치입니다.
- `GraphType`: 간선 리스트로 그래프를 나타냅니다.
 - `n`: 그래프의 간선 수입니다.
 - `edges[2 * MAX_VERTICES]`: `Edge` 구조체 배열입니다.

- **핵심 함수 (DSU 연산):**

- `void set_init(int n)`: 각 정점의 부모를 -1로 설정하여 DSU 구조를 초기화합니다(각 정점은 초기에 자신만의 집합에 속함). 여기서 `n`은 정점의 수를 의미해야 하지만, `kruskal` 함수 내에서는 간선의 수인 `g->n`으로 호출됩니다. 이는 크루스칼 알고리즘에 대해 정점의 수로 초기화되어야 하는 잠재적인 버그로 보입니다.
- `int set_find(int curr)`: 경로 압축을 사용하여 (암시적으로 위로 반복하여) 정점 `curr`를 포함하는 집합의 대표(루트)를 찾습니다.
- `void set_union(int a, int b)`: 정점 `a`와 `b`를 포함하는 집합이 이미 같은 집합이 아니라면 한 루트가 다른 루트를 가리키도록 하여 병합합니다.

- **핵심 함수 (그래프 및 크루스칼):**

- `void graph_init(GraphType* g)`: 간선 수 `g->n`을 0으로 설정하고 간선 가중치를 `INF`로 초기화하여 그래프 구조를 초기화합니다.
- `void insert_edge(GraphType* g, int start, int end, int w)`: `g->edges` 배열에 새 간선을 추가하고 `g->n`을 증가시킵니다.
- `int compare(const void* a, const void* b)`: `qsort`에 필요한 비교 함수로, 간선을 가중치에 따라 오름차순으로 정렬합니다.
- `void kruskal(GraphType *g)`: 크루스칼 알고리즘을 구현합니다.
 1. `set_init(g->n)`을 사용하여 DSU 구조를 초기화합니다 (이상적으로는 `set_init(NUM_VERTICES)`여야 합니다).
 2. `qsort`를 사용하여 `g->edges`의 모든 간선을 가중치에 따라 비내림차순으로 정렬합니다.
 3. `edge_accepted = 0`으로 초기화합니다.
 4. 정렬된 간선을 반복합니다:
 - 현재 간선 `e = (u, v)`와 가중치 `w`에 대해:
 - `u`를 포함하는 집합(`uset = set_find(e.start)`)과 `v`를 포함하는 집합(`vset = set_find(e.end)`)을 찾습니다.
 - `uset != vset`이면 (즉, 이 간선을 추가해도 사이클이 형성되지 않으면):
 - 간선이 선택되었음을 출력합니다.

- `edge_accepted`를 증가시킵니다.
 - `set_union(uset, vset)`을 사용하여 `u`와 `v`의 집합을 병합합니다.
- 루프는 `edge_accepted`가 `NUM_VERTICES - 1`과 같아질 때까지 (또는 모든 간선이 처리될 때까지) 계속됩니다.
- `int main()`:
 - `GraphType` 구조체에 대한 메모리를 동적으로 할당합니다.
 - `graph_init()`을 사용하여 그래프를 초기화합니다.
 - 여러 간선과 해당 가중치를 삽입합니다. `main` 함수는 암묵적으로 7개의 정점(0-6)을 정의합니다.
 - `kruskal(g)`을 호출합니다.
 - 할당된 그래프 메모리를 해제합니다.
 - 주의: `set_init(g->n)`은 `kruskal` 내에서 호출되는데, 여기서 `g->n`은 간선의 수입니다. 이는 정점의 수로 초기화되어야 합니다. 또한 루프 조건 `edge_accepted < (g->n - 1)`도 정점 수 대신 간선 수 `g->n`을 사용합니다. 이는 V개의 정점을 가진 MST에 대해 V-1개의 간선이 선택되어야 하므로 수정되지 않으면 잘못된 MST 결과 또는 런타임 오류가 발생할 수 있습니다.

구현된 알고리즘:

최소 신장 트리를 찾기 위한 크루스칼 알고리즘입니다. 이는 탐욕 알고리즘으로, 모든 간선을 가중치별로 정렬하고 사이클을 형성하지 않는 경우 간선을 하나씩 추가하며, DSU 구조를 사용하여 사이클을 효율적으로 감지합니다.

코드 내 예시: `main` 함수는 그래프를 간선 리스트 형태로 초기화하고 `kruskal` 함수를 호출하여 MST를 찾는 과정을 보여줍니다.

4. Prim.c Prim (프림) 알고리즘

이 파일은 **프림 알고리즘**을 구현합니다. 이 알고리즘 역시 연결된, 무방향, 가중 그래프에 대한 최소 신장 트리 (MST)를 찾는 데 사용됩니다.

주요 구성 요소:

- 자료구조:
 - `GraphType`: 인접 행렬을 사용하여 그래프를 나타냅니다.
 - `n`: 정점의 수입니다.
 - `weight[MAX_VERTICES][MAX_VERTICES]`: 간선 가중치를 저장하는 인접 행렬입니다. `INF(1000L)`는 직접적인 경로가 없거나 매우 큰 가중치를 나타냅니다.
 - `selected[MAX_VERTICES]`: 전역 불리언(정수) 배열입니다. `selected[i]`가 `TRUE`이면 정점 `i`가 MST에 포함되었음을, `FALSE`이면 그렇지 않음을 의미합니다.
 - `distance[MAX_VERTICES]`: 전역 정수 배열입니다. `distance[v]`는 (아직 MST에 포함되지 않은) 정점 `v`를 이미 MST에 포함된 어떤 정점과 연결하는 간선의 최소 가중치를 저장합니다.
- 핵심 함수:
 - `int get_min_vertex(int n)`: 아직 `selected`되지 않았으면서 가장 작은 `distance` 값을 가진 정점 `v`를 찾아 반환합니다.
 - `void prim(GraphType* g, int s)`: 시작 정점 `s`에서 프림 알고리즘을 구현하는 주 함수입니다.

1. 모든 정점 u 에 대해 $distance[u]$ 를 INF 로 초기화합니다.
 2. $distance[s]$ 를 0으로 설정합니다 (시작 정점은 초기에 비용 0으로 "도달 가능"한 유일한 정점임).
 3. $g \rightarrow n$ 번 반복합니다 (MST를 위해 $g \rightarrow n$ 개의 정점을 선택하기 위함):
 - $get_min_vertex(g \rightarrow n)$ 를 호출하여 아직 $selected$ 에 포함되지 않았고 연결 비용 ($distance[u]$)이 최소인 정점 u 를 선택합니다.
 - u 를 선택된 것으로 표시합니다: $selected[u] = TRUE$.
 - $distance[u] == INF$ 이면 나머지 정점은 도달할 수 없음을 의미하므로(그래프가 연결되지 않음) 함수가 반환될 수 있습니다.
 - 정점 u 가 MST에 추가되었음을 출력합니다.
 - u 에 인접한 모든 정점 v 에 대해:
 - v 가 아직 $selected$ 되지 않았고 간선 (u, v) 의 가중치($g \rightarrow weight[u][v]$)가 현재 $distance[v]$ 보다 작으면 $distance[v]$ 를 $g \rightarrow weight[u][v]$ 로 업데이트합니다.
- `int main():`
 - 7개의 정점과 미리 정의된 간선 가중치를 가진 `GraphType` 구조체 g 를 초기화합니다.
 - 정점 0에서 시작하는 MST를 찾기 위해 `prim(&g, 0)`를 호출합니다.

구현된 알고리즘:

프림 알고리즘입니다. 이는 탐욕 알고리즘으로, 시작 정점에서 MST를 확장해 나갑니다. 각 단계에서 MST에 있는 정점과 아직 MST에 없는 정점을 연결하는 가장 저렴한 간선을 추가합니다.

코드 내 예시: `main` 함수는 인접 행렬로 표현된 그래프를 초기화하고, 특정 정점(0번)에서 시작하여 프림 알고리즘으로 MST를 구성하는 예시를 보여줍니다.

5. `TopoSort.c` Topological Sort (위상 정렬)

이 파일은 방향 비순환 그래프(DAG)에 대한 **위상 정렬**을 구현합니다. 위상 정렬은 정점들의 선형 순서로, 모든 방향 간선 (u, v) (정점 u 에서 v 로)에 대해 정점 u 가 정점 v 보다 순서상 앞에 오도록 하는 것입니다. 이 구현은 칸(Kahn) 알고리즘(진입 차수 기반)을 사용합니다.

주요 구성 요소:

- **자료구조 (그래프):**
 - `GraphNode`: 인접 리스트의 노드를 나타내는 구조체입니다.
 - `vertex`: 정점 번호입니다.
 - `link`: 인접 리스트의 다음 `GraphNode`를 가리키는 포인터입니다.
 - `GraphType`: 인접 리스트 배열을 사용하여 그래프를 나타냅니다.
 - `n`: 현재 그래프의 정점 수입니다.
 - `adj_list[MAX_VERTICES]`: `GraphNode`에 대한 포인터 배열로, `adj_list[i]`는 정점 i 의 이웃 리스트의 헤드를 가리킵니다.
- **자료구조 (스택 - 칸 알고리즘용, 큐가 더 일반적임):**
 - `StackType`: 간단한 배열 기반 스택입니다.
 - `stack[MAX_STACK_SIZE]`: 스택 원소를 저장하는 배열입니다.
 - `top`: 최상위 원소의 인덱스입니다.

- 스택 유틸리티 함수: `init()`, `is_empty()`, `is_full()`, `push()`, `pop()`.

- 핵심 함수 (그래프):

- `void graph_init(GraphType *g):` `g->n`을 0으로 설정하고 모든 인접 리스트 헤드를 `NULL`로 설정하여 그래프를 초기화합니다.
- `void insert_vertex(GraphType *g, int v):` 정점 수 `g->n`을 증가시킵니다.
- `void insert_edge(GraphType *g, int u, int v):` `u`에서 `v`로 가는 방향 간선을 추가합니다. `v`에 대한 새 `GraphNode`를 만들어 `g->adj_list[u]`의 시작 부분에 삽입합니다.

- 핵심 함수 (위상 정렬):

- `int topo_sort(GraphType *g):` 위상 정렬을 위한 칸 알고리즘을 구현합니다.
 1. `g->n` 크기의 `in_degree` 배열을 동적으로 할당합니다.
 2. 모든 `in_degree[i]`를 0으로 초기화합니다.
 3. 각 정점의 진입 차수를 계산합니다: 각 정점 `i`에 대해 인접 리스트(`g->adj_list[i]`)를 순회합니다. 각 이웃 `node->vertex`에 대해 `in_degree[node->vertex]`를 증가시킵니다.
 4. 빈 스택 `s`를 초기화합니다.
 5. `in_degree[i] == 0`인 모든 정점 `i`를 스택에 추가합니다.
 6. 스택이 비어 있지 않은 동안:
 - 스택에서 정점 `w`를 꺼냅니다.
 - 정점 `w`를 출력합니다 (이것이 위상 순서의 일부임).
 - `w`의 각 이웃 `u`에 대해 (`g->adj_list[w]`에서 얻음):
 - `in_degree[u]`를 감소시킵니다.
 - `in_degree[u]`가 0이 되면 `u`를 스택에 넣습니다.
 7. `in_degree` 배열을 해제합니다.
 8. (`i == g->n`)을 반환합니다 (여기서 `i`는 진입 차수 계산/초기화 단계의 루프 카운터이며, 정렬된 정점의 수가 아님). 이 반환 조건은 정렬 성공 여부(예: 사이클 감지)를 확인하는 데 부정확해 보입니다. 성공적인 정렬은 모든 `g->n`개의 정점을 처리합니다. 꺼낸 정점의 카운터를 `g->n`과 비교해야 합니다.

- `int main():`

- `GraphType` 구조체 `g`를 초기화합니다.
- 6개의 정점(0~5)을 삽입합니다.
- DAG를 형성하기 위해 여러 방향 간선을 삽입합니다.
- 위상 정렬을 수행하고 출력하기 위해 `topo_sort(&g)`를 호출합니다.

구현된 알고리즘:

위상 정렬을 위한 칸(Kahn) 알고리즘입니다. 작동 방식은 다음과 같습니다:

1. 모든 정점의 진입 차수를 계산합니다.
2. 진입 차수가 0인 모든 정점으로 큐(또는 여기서는 스택)를 초기화합니다.
3. 큐/스택이 비어 있지 않은 동안:
 - _ 정점 `u`를 디큐/팝하고 위상 정렬에 추가합니다. `u`의 각 이웃 `v`에 대해: `v`의 진입 차수를 감소시킵니다. `v`의 진입 차수가 0이 되면 `v`를 인큐/푸시합니다. 위상 정렬의 정점 수가 그래프의 총 정점 수와 같지 않으면 그래프에 사이클이 있는 것입니다. 제공된 코드의 반환 값 로직은 이를 정확하게 반영하도록 조정될 필요가 있습니다.

코드 내 예시: `main` 함수는 방향 그래프를 생성하고, 정점과 간선을 추가한 후 `topo_sort` 함수를 호출하여 위상 정렬 순서를 출력하는 예시를 보여줍니다.
